A Novel 8-bit Quantization for Large Language Model Weights Delivers 3.7× Less Mean-Square-Error

Heath Hunnicutt*
Ruach Tov Collective
heath@ruachtov.ai

Mavchin†
Ruach Tov Collective
agents@ruachtov.ai

Iyun‡
Ruach Tov Collective
agents@ruachtov.ai

June 2026

Abstract

We observe that normalized transformer weights concentrate 56% of values in the lowest quartile of their dynamic range. Standard Q8_0 quantization divides resolution uniformly across this range, allocating precision to sparse tail regions. We propose **Prefix-VBR**, a fast variable bit-rate scheme that allocates 7 bits to the dense near-zero region and progressively fewer bits to sparser ranges. On 25.2 million weight values from a production embedding model, Prefix-VBR achieves **3.69× lower MSE** and **+5.7 dB SNR** compared to Q8_0. End-to-end validation on a 600M-parameter language model confirms that the MSE improvement produces 2× **better logit fidelity** versus the unquantized reference (KL divergence 6.6×10^{-4} vs. 1.15×10^{-3} nats). Across 100 diverse prompts, Prefix-VBR matches the unquantized reference on 99 top-1 predictions vs. Q8_0's 96. The dequantization computation requires only bit masking and multiplication, and trades a 1.27× throughput cost for 3.7× less quantization noise power.

1 Introduction

Quantization is the dominant technique for reducing memory and compute requirements in large language model inference. The widely-used Q8_0 format [1] quantizes each weight to an 8-bit signed integer with a 16-bit floating-point scale factor shared per block of 32 weights. This uniform quantization of the scale factor implicitly assumes that weights are distributed approximately uniformly within each block's dynamic range.

We show that this assumption is suboptimal. After per-block normalization, transformer weights exhibit a strongly non-uniform distribution: 56% of values fall in the lowest quartile of the range. This concentration near zero means that uniform quantization wastes most of its resolution in regions where few weights reside.

We propose **Prefix-VBR** (Prefix Variable Bit-Rate), an 8-bit packed format that uses a unary prefix code to allocate more mantissa bits to the dense near-zero region. The key idea is simple: where the data is dense, use more bits; where it is sparse, use fewer. The prefix encoding achieves this within a fixed 8-bit budget per weight, with no change to the per-block scale factor or storage layout.

2 Weight Distribution Analysis

We analyzed 25,182,720 weight values from the granite-embedding model (384 dimensions, 65,580 WordNet vocabulary entries). Table 1 shows the distribution of per-block normalized magni-

*Corresponding human author.
†Corresponding Claude Opus 4.6 author.
‡Claude Opus 4.7.

tudes.

Table 1: Distribution of per-block normalized weight magnitudes ($|w_{\text{norm}}|$ after dividing by block maximum / 127).

Range	Fraction of weights	Cumulative
$[0, 32)$	55.9%	55.9%
$[32, 64)$	27.4%	83.3%
$[64, 96)$	10.0%	93.3%
$[96, 127]$	6.7%	100.0%

The median magnitude is 27.6 (within the first quartile), and the 90th percentile is 81.3. This extreme concentration motivates non-uniform quantization: allocating 4× more levels to the $[0, 32)$ range captures the majority of the distribution with high fidelity.

3 Prefix-VBR Format

Each weight is encoded as 8 bits: 1 sign bit and 7 bits of prefix-encoded magnitude. The prefix tree partitions the $[0, 127)$ range into four segments with decreasing resolution:

Table 2: Prefix-VBR encoding. The unary prefix determines the segment; remaining bits encode the position within the segment.

Prefix	Layout	Range	Levels	Step size
`0`	`0xxxxxxx`	$[0, 32)$	128	0.25
`10`	`10xxxxxx`	$[32, 64)$	64	0.50
`110`	`110xxxxx`	$[64, 96)$	32	1.00
`1110`	`1110xxxx`	$[96, 127)$	16	1.94

Storage is identical to Q8_0: each weight occupies exactly 1 byte, and each block of 32 weights shares a 16-bit (FP16) scale factor. The total storage cost is 8.5 bits per weight, unchanged from Q8_0.

Prefix-VBR design is based on our observation that transformer weight distributions are analogous to audio signal distributions, where companding laws (such as μ-law) allocate resolution proportionally to signal density. The prefix-VBR scheme achieves a similar effect with a computationally simpler encoding that avoids logarithmic or exponential operations.

4 Dequantization Kernel

The dequantization operation requires only bit masking and a single multiply per weight:

```
__device__ float dequant_vbr(uint8_t code, float scale) {
    float sign = (code & 0x80) ? -1.0f : 1.0f;
    uint8_t mag = code & 0x7F;
    float val;
    if (!(mag & 0x40))        // 0xxxxxxx: [0, 32)
        val = mag * (32.0f / 127.0f);
    else if (!(mag & 0x20))   // 10xxxxxx: [32, 64)
        val = 32.0f + (mag & 0x3F) * (32.0f / 63.0f);
    else if (!(mag & 0x10))   // 110xxxxx: [64, 96)
        val = 64.0f + (mag & 0x1F) * (32.0f / 31.0f);
    else                      // 1110xxxx: [96, 127)
```

```
        val = 96.0f + (mag & 0x0F) * (31.0f / 15.0f);
    return sign * val * scale;
}
```

No lookup tables or transcendental function evaluations are required. The branch on the prefix bits is well-predicted: the first branch (`0xxxxxxx`) is taken for 56% of weights, yielding high branch-prediction accuracy on GPU warp-level execution.

5 Experimental Results

We compared three quantization schemes, all using identical per-block scaling (block size 32, scale = max(|block|)/127).

Table 3: Quantization quality comparison. All schemes use per-block scaling with block size 32 and 8 bits per weight. Data: 25.2M granite-embedding weight values.

Scheme	MSE	SNR (dB)	vs. Q8_0	quality
Prefix-VBR	3.08×10^{-8}	49.3	**3.69**×	**better**
Q8_0 (baseline)	1.14×10^{-7}	43.6	1.00×	
μ-law ($\mu = 15$)	1.05×10^{-7}	43.9	1.08×	
μ-law ($\mu = 255$)	3.08×10^{-7}	39.2	0.37×	worse

5.1 Discussion

The μ-law result is informative. At low μ (mild companding), μ-law barely outperforms Q8_0 (1.08× at $\mu = 15$). At higher μ values, performance degrades because the logarithmic companding curve over-compresses the near-zero region relative to the actual weight distribution. Prefix-VBR avoids this by using a piecewise linear mapping with segment boundaries tuned to the observed distribution.

5.2 End-to-End Model Validation

To verify that improved weight MSE translates to improved generation quality, we round-trip quantized all weight matrices of a Qwen3-0.6B model using both Q8_0 and Prefix-VBR, then compared outputs against the unquantized BF16 reference.

Token match. Over 1,000 tokens of autoregressive greedy decoding, Prefix-VBR matches the unquantized reference **token-for-token with zero divergence** (1005/1005 tokens, first divergence: none). Q8_0 also achieves 100% token match. Both quantizations are sound for generation.

Logit fidelity. Comparing full-vocabulary logit vectors against the unquantized reference reveals a clearer separation:

Table 4: Logit fidelity vs. unquantized BF16 reference (Qwen3-0.6B, single forward pass, full vocabulary).

Scheme	Mean $\lvert\Delta\text{logit}\rvert$	Max $\lvert\Delta\text{logit}\rvert$	KL (nats)	Pearson r
Q8_0	0.0650	0.3872	1.15×10^{-3}	0.99969
Prefix-VBR	**0.0333**	**0.1856**	$\mathbf{6.56 \times 10^{-4}}$	**0.99991**

Prefix-VBR is approximately 2× closer to the unquantized model on every metric: mean logit deviation (0.033 vs. 0.065), maximum deviation (0.186 vs. 0.387), KL divergence (6.6×10^{-4} vs. 1.15×10^{-3} nats), and Pearson correlation (0.99991 vs. 0.99969). Better weight MSE genuinely results in better logit fidelity end-to-end. We note that KL divergence from the unquantized reference is the same metric used by the llama.cpp/GGUF ecosystem to evaluate its own quantization types, positioning our comparison within established practice.

Perplexity. As a standard point of comparison, we report WikiText-2 perplexity:

Table 5: WikiText-2 perplexity (Qwen3-0.6B, 4088 tokens, context length 512).

Scheme	bits/wt	PPL	Δ vs. FP16
FP16 (reference)	16	30.900	—
Q8_0	8.5	30.973	+0.073
Prefix-VBR	8.5	**30.969**	**+0.068**

Both quantizations produce near-lossless perplexity (< 0.3% relative increase), with Prefix-VBR approximately 7% closer to the FP16 reference than Q8_0 at identical bit rate.

Dequantization throughput. The prefix-encoded dequantization incurs a throughput cost relative to Q8_0's branchless multiply: 84.2 GB/s vs. 107.1 GB/s on a Tesla P4, a factor of 1.27× slower. However, this cost is amortized under the dominant matrix multiplication and represents a design trade: a 1.27× linear slowdown on the cheap dequantization operation buys a 3.7× reduction in quantization noise power (+5.7 dB), which propagates through the entire forward pass.

5.3 Adherence Across Diverse Prompts

To test robustness beyond a single prompt, we evaluated both schemes on 100 diverse prompts drawn from two maximally different registers: recipe prose and `comp.lang.c` Usenet idiom (e.g., `sizeof`, undefined behavior, null pointer semantics). For each prompt, we compared the full-vocabulary logit vector and greedy argmax against the unquantized BF16 reference.

Table 6: Adherence over 100 diverse prompts (Qwen3-0.6B vs. unquantized reference).

Scheme	Mean $\lvert\Delta\text{logit}\rvert$	Mean KL (nats)	Top-1 match	Closer to gold
Q8_0	0.0586	1.42×10^{-3}	96/100	1/100
Prefix-VBR	**0.0298**	$\mathbf{3.91 \times 10^{-4}}$	**99/100**	**99/100**

Prefix-VBR is closer to the unquantized reference on 99 of 100 prompts by logit deviation and 95 of 100 by KL divergence, with zero ties. The advantage is stable across both text registers, confirming that the fidelity improvement is a property of the quantization scheme, not the text distribution. The ratio (1.97× lower mean logit deviation) is consistent with the single-prompt measurement (2.0×), confirming the 16-prompt pilot was not a small-sample artifact.

5.4 Interlinear Text Exhibit

Across the 100 prompts, Prefix-VBR's greedy argmax diverged from gold on 1 prompt; Q8_0 diverged on 4. Figure 1 shows all five divergence cases in interlinear format, with the divergent token marked by *.

```
[p72] VBR diverges - '...before you roll'
 gold |                              roll  it    ,      and
 q8   |                              roll  it    ,      and
 vbr  |                              roll  the*  dough* ,*

[p59] Q8_0 diverges - '...handful of grated'
 gold |                               grated  Parm    esan  cheese
 q8   |                               grated  cheese* .*    The*
 vbr  |                               grated  Parm    esan  cheese

[p66] Q8_0 diverges - '...until the'
 gold |                   until  the  water  is  clean
 q8   |                   until  the  pan*   is  clean
 vbr  |                   until  the  water  is  clean

[p67] Q8_0 diverges - '...absorb the'
 gold |                   absorb  the  broth       .  Then
 q8   |                   absorb  the  vegetables* .  Then
 vbr  |                   absorb  the  broth       .  Then

[p81] Q8_0 diverges - '...and the standard'
 gold |                        the  standard  way      to  handle
 q8   |                        the  standard  library* 's* string*
 vbr  |                        the  standard  way      to  handle
```

Figure 1: All divergence cases from the 100-prompt evaluation. VBR's single divergence (p72) is a benign function-word substitution ("it" → "the dough"); Q8_0's four divergences include content-word errors ("Parmesan" → "cheese", "water" → "pan", "broth" → "vegetables") and one case (p81) that degenerates into repetition under extended free-running.

5.5 Negative Result: Dense-120

A single-branch variant allocating 120 uniform levels to the dense region and 7 to the tail was tested as a minimal-complexity alternative. It did **not** beat Q8_0 (best 0.80×). The reason: per-block max-scaling places every block's maximum at the top of the range, so a uniform-dense scheme either starves the tail or converges to Q8_0. Beating Q8_0 requires super-uniform density near zero, which requires variable resolution across multiple bands.

6 Relation to Prior Work

Information-theoretic quantization has a long history. Lloyd-Max quantizers [2] minimize MSE for a given number of levels, but require per-distribution optimization. Companding laws (μ-law, A-law) from telecommunications [3] provide fixed nonlinear mappings but are optimized for speech signals, not neural network weights.

Recent work on neural network quantization has focused on mixed-precision schemes [4, 5], where different layers receive different bit widths, and non-uniform quantization via learned mappings. NUPES [7] uses power-function automorphisms with learned exponents; LCQ [8] learns companding functions for 2–4 bit models. Both require training or optimization of the quantization parameters. Prefix-VBR is orthogonal: it is a fixed, analytical format that requires no learning, and improves precision *within* a fixed 8-bit budget by redistributing resolution according to the weight distribution.

The DFloat11 format [6] achieves lossless 30% compression of BF16 weights with bit-identical output. Prefix-VBR operates in the lossy regime but at a much higher compression ratio (32-bit

to 8-bit), and achieves its gains through distribution-aware level allocation rather than entropy coding.

7 Limitations and Future Work

1. **Model scale.** Our end-to-end validation uses a 0.6B parameter model. Validation on larger models (7B, 27B) is needed to confirm the logit fidelity improvement scales with model size.

2. **Dequantization throughput.** The 1.27× slowdown from prefix branching may be reducible through warp-uniform optimization or branchless bit-manipulation equivalents. The current implementation is naïve.

3. **Optimal boundaries.** The segment boundaries $[0, 32, 64, 96, 127]$ were hand-tuned from the observed distribution. Optimization over boundary placement for a given weight distribution could yield further improvements.

4. **Fixed-width output.** The prefix-VBR scheme uses a fixed-width output (8 bits per weight). A variable-width, arithmetic coding approach could approach the information-theoretic optimum at the cost of more complex decoding. The trade-off between compression efficiency and decoding throughput on GPU is worth investigating.

5. **Degeneration resistance.** In extended free-running on prompt p81, Q8_0 collapses into degenerate repetition while Prefix-VBR remains coherent. Whether this qualitative robustness advantage generalizes across model families and scales is a promising direction for future investigation.

6. **Per-layer adaptation.** Different tensor types (attention projections vs. MLP gates) show different weight distributions. Per-tensor-type boundary optimization could improve results further.

8 Conclusion

Prefix-VBR achieves 3.69× lower weight MSE and approximately 2× better logit fidelity than Q8_0 at identical storage cost, by putting more quantization levels where more weights are. Over 1,000 tokens of autoregressive generation, Prefix-VBR's logit vectors are measurably closer to the original on every metric (mean deviation, KL divergence, correlation).

The dequantization kernel trades 1.27× throughput for 3.7× less quantization noise power (+5.7 dB SNR)—a design that spends a little speed where it is cheap to buy a lot of fidelity where it is valuable. A negative result (Dense-120) confirms that the multi-band prefix structure is worthwhile complexity to beat uniform quantization under per-block scaling.

The format requires no lookup tables, no transcendental functions, and no changes to the per-block scaling infrastructure that Q8_0 already provides. It is a drop-in replacement that improves precision at a modest and amortizable throughput cost.

Reproducibility

Code and data are available at: `https://github.com/Ruach-Tov/LlamaTov`

References

[1] Georgi Gerganov, *ggml: Tensor library for machine learning*, 2023. `https://github.com/ggerganov/ggml`

[2] S.P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inform. Theory*, 1982.

[3] ITU-T G.711, "Pulse Code Modulation (PCM) of Voice Frequencies," 1988.

[4] E. Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers," *ICLR*, 2023.

[5] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration," *MLSys*, 2024.

[6] DFloat11, "70% Size, 100% Accuracy: Lossless LLM Compression," *arXiv*, 2025.

[7] E. Yvinec, A. Dapogny, K. Bailly, "NUPES: Non-Uniform Post-Training Quantization via Power Exponent Search," *arXiv:2308.05600*, 2023.

[8] K. Yamamoto, "Learnable Companding Quantization for Accurate Low-bit Neural Networks," *CVPR*, 2021.

A Reproducibility: Sources and Methods

All programs listed below are SWI-Prolog. The input data (25,182,720 FP16 values, 48 MB) and all program listings are included in the repository at `Papers/2026/June/Prefix-VBR/`.

A.1 Table 1: Weight Distribution

Table 1 is produced by the following program, which reads the granite-embedding vectors, applies per-block normalization, and computes the distribution histogram.

Data extraction.

```
psql -d claude_conversations -t -A -c \
  "SELECT␣replace(replace(
␣␣␣␣␣embedding::text,'[',''),']','')
␣␣␣FROM␣wordnet_embeddings_granite" \
  > embeddings.csv
```

Analysis.

```
:- use_module(library(readutil)).
:- use_module(library(lists)).

block_size(32).

main :-
    read_all_floats('embeddings.csv', W),
    per_block_normalize(W, 32, Norm),
    Bins = [0-32, 32-64, 64-96, 96-128],
    compute_histogram(Norm, Bins, Table1),
    print_results(Table1, 0.0).

per_block_normalize(Weights, BS, Norm) :-
    chunk_list(Weights, BS, Blocks),
    include(full_block(BS), Blocks, Full),
```

```
    maplist(normalize_block, Full, NBlocks),
    append(NBlocks, Norm).

normalize_block(Block, NormBlock) :-
    maplist([W,A]>>(A is abs(W)), Block, Abs),
    max_list(Abs, MaxAbs),
    Scale is MaxAbs / 127.0,
    maplist({Scale}/[W,NW]>>(NW is W/Scale),
            Block, NormBlock).

compute_histogram(Values, Bins, Results) :-
    length(Values, Total),
    maplist(count_bin(Values, Total),
            Bins, Results).

count_bin(Vals, Total, Lo-Hi, bin(Lo,Hi,Pct)) :-
    include({Lo,Hi}/[V]>>(
        A is abs(V), A >= Lo, A < Hi),
        Vals, In),
    length(In, Count),
    Pct is (Count / Total) * 100.
```

Running this program over all 65,580 embeddings (25,182,720 values) produces the distribution in Table 1.

A.2 Table 3: MSE Comparison

Table 3 is produced by the following program, which quantizes the same embeddings with Q8_0, Prefix-VBR, and μ-law, then measures round-trip reconstruction MSE for each scheme. All schemes use identical per-block scaling (block size 32).

```
main :-
    read_all_floats('embeddings.f16', W),
    chunk_list(W, 32, Blocks),

    maplist(q8_0_roundtrip, Blocks, Q8R),
    maplist(prefix_vbr_roundtrip, Blocks, VBRR),
    maplist(mulaw_roundtrip(15), Blocks, MuR),

    flatten(Blocks, Orig),
    compute_mse(Orig, Q8R,  MSE_Q8),
    compute_mse(Orig, VBRR, MSE_VBR),
    compute_mse(Orig, MuR,  MSE_Mu),
    print_table(MSE_Q8, MSE_VBR, MSE_Mu).

q8_0_roundtrip(Block, Recovered) :-
    max_abs(Block, MaxAbs),
    Scale is MaxAbs / 127.0,
    maplist({Scale}/[W, R]>>(
        Q is max(-127, min(127, round(W/Scale))),
        R is Q * Scale), Block, Recovered).

prefix_vbr_roundtrip(Block, Recovered) :-
    max_abs(Block, MaxAbs),
    Scale is MaxAbs / 127.0,
    maplist({Scale}/[W, R]>>(
        Norm is abs(W / Scale),
        vbr_encode_decode(Norm, Rec),
```

```
        R is sign(W) * Rec * Scale),
        Block, Recovered).

vbr_encode_decode(N, R) :-      % [0, 32)
    N < 32, !,
    Q is round(N * 127.0 / 32.0),
    R is Q * 32.0 / 127.0.
vbr_encode_decode(N, R) :-      % [32, 64)
    N < 64, !,
    Q is round((N-32) * 63.0 / 32.0),
    R is 32.0 + Q * 32.0 / 63.0.
vbr_encode_decode(N, R) :-      % [64, 96)
    N < 96, !,
    Q is round((N-64) * 31.0 / 32.0),
    R is 64.0 + Q * 32.0 / 31.0.
vbr_encode_decode(N, R) :-      % [96, 127)
    Q is round((N-96) * 15.0 / 31.0),
    R is 96.0 + Q * 31.0 / 15.0.
```

The full executable program, including the μ-law implementation and FP16 binary reader, is included in the repository as `reproduce_table2_mse.pl`.

A.3 Table 4: Logit Fidelity

Table 4 compares full-vocabulary logit vectors from Q8_0 and Prefix-VBR quantized models against the unquantized BF16 reference. The measurement script round-trip quantizes all weight tensors in memory, runs a single forward pass on a fixed prompt, and computes mean $|\Delta\text{logit}|$, max deviation, KL divergence, and Pearson correlation over the full vocabulary. Repository: `Papers/2026/June/Prefix-VBR/prefix_vbr_logits.py`.

A.4 Table 5: Perplexity

Table 5 reports WikiText-2 perplexity for FP16, Q8_0, and Prefix-VBR on Qwen3-0.6B (4,088 tokens, context length 512). The measurement uses a single-pass negative log-likelihood computation over the test corpus. Repository: `Papers/2026/June/Prefix-VBR/ppl_gpu.py`.

A.5 Table 6: 100-Prompt Adherence

Table 6 evaluates top-1 argmax agreement with the unquantized reference across 100 diverse prompts drawn from two registers (recipe prose and `comp.lang.c` Usenet idiom). For each prompt, the script runs a single forward pass under Q8_0 and Prefix-VBR quantization, compares the full-vocabulary logit vector against the BF16 reference, and records which scheme is closer to gold. Repository: `Papers/2026/June/Prefix-VBR/prefix_vbr_adherence100.py`.